

DAV Public School, Haldia
Senior Secondary Unit

Computer Science Project
on
MySQL Connectivity



2025-2026

In the partial fulfillment of Senior School Certificate
Examination for the session 2025-2026 conducted by
Central Board of Secondary Education

Utsav Maity
CLASS – XII

CBSE Reg. No. :

CBSE Roll. No. :

DAV Public School, Haldia

Department of Computer Science

CERTIFICATE



This is to certify that Utsav Maity of Class XII A of DAV Public School, Haldia, has successfully completed the Course Project on "Controlled Menu-Driven MySQL Connectivity" under the guidance of the subject teacher, Mr. Mukesh Patra during the academic year 2025-2026 in partial fulfillment of the Computer Science Practical Examination (SSCE - 2026) conducted by C.B.S.E., New Delhi.

Signature of External
Examiner

Signature of Computer Science
Faculty

1 Acknowledgement

I would like to express my sincere gratitude to all those who have supported and guided me throughout this project. I am particularly thankful to our esteemed Principal, Mr. N. P. Dutta, for fostering an environment of academic excellence and providing the necessary resources. My heartfelt appreciation goes to my computer teacher, Mr. Mukesh Patra, whose expert guidance, valuable insights, and constant encouragement were instrumental in navigating the project's challenges. I am also deeply indebted to my parents for their unwavering support, endless patience, and steadfast belief in my abilities. This accomplishment would not have been possible without the collective support and encouragement from each one of them, which ultimately led to the successful completion of this project.

Signature:

Name: Utsav Maity

Class: XII A

Academic Year: 2025-2026

Contents

1	Introduction	1
2	Objective of Project	2
3	Introduction to Tools Used	3
4	Database Structure	4
5	Game_Database.py Implimentation Details	5
6	Source Code – Game_Database.py	10
7	Output (Code Run)	18
8	Conclusion	23
9	Bibliography	24

Introduction

This project implements a professional database management system for digital platform administration, demonstrating real-world applications in user account management and financial transaction processing. While presented through a gaming context, the core system mirrors technologies used across e-commerce platforms and gaming industries for handling user registrations, secure authentication, currency management, and financial operations. The project showcases essential database concepts including relational design, foreign key integrity, transaction processing, and data persistence through backup systems. These skills are directly transferable to commercial systems handling customer data, loyalty programs, and financial transactions across various digital platforms.

Real Life Application :

The database architecture developed in this project serves fundamental purposes across multiple digital domains:

In the gaming industry, this system forms the backbone for player account management, virtual currency handling, and in-game transaction processing. It supports the core operations of digital gaming platforms by maintaining user profiles, managing purchase histories, and tracking virtual economies.

For e-commerce platforms, this structure enables essential customer management and transaction processing capabilities. The system handles user registration, product catalog management, and financial operations that are crucial for online retail environments, while supporting loyalty programs and digital wallet functionalities.

KEY FEATURES :

- User account management with secure validation
- Financial transaction processing and tracking
- Currency conversion systems
- Revenue calculation and reporting
- Data backup and recovery mechanisms
- Professional error handling and input validation

This integrated approach demonstrates how core database management principles form the foundation for both entertainment and commercial digital platforms, providing scalable solutions for user management and financial operations in today's digital economy.

Objective of project

1. To Design and Develop a Menu-Driven Database System

- Create an application with a clear, multi-level menu for easy user navigation.
- Build an interface that cleanly separates data management from financial operations.

2. To Implement Core Database Operations (CRUD)

- Let users add new player profiles with all required information.
- Allow users to search for and display any player's complete record.
- Enable the editing of player details like username or password securely.
- Permanently remove a player's record only after confirming the action.

3. To Integrate Financial Transaction Features

- Process purchases of in-game currency using real-money values.
- Allow players to exchange one type of currency for another.
- Record coins and gems earned by players through gameplay.
- Maintain a log of all money spent by each player.
- To Calculate the Revenue generated through Transactions.

4. To Ensure Data Integrity and Security

- Validate all entries for correct format, type, and length.
- Restrict sensitive functions like database reset with a password.
- Save the entire database to a file for safekeeping.
- Reload the database from a backup file after data loss.

Introduction to Tools Used

The development of this database management system leverages a carefully selected set of software tools and technologies, each serving a specific purpose in the application's architecture and functionality.

MySQL Version 8.0.42 serves as the foundational database management system, providing robust data storage, retrieval, and transaction processing capabilities. This relational database management system from Oracle Corporation handles all persistent data storage, including player profiles, financial transactions, and inventory management. Its advanced features such as foreign key constraints, cascade operations, and transaction integrity ensure data consistency and reliability throughout all operations.

Python Version 3.12 functions as the primary programming language, chosen for its extensive library support and database connectivity features. The Python Software Foundation's latest stable release provides the backbone for implementing business logic, user interface management, and system integration. Its object-oriented capabilities enable the creation of modular code structure, while its exception handling mechanisms ensure robust error management throughout the application.

mysql-connector-python Version 9.4.0 acts as the critical bridge between the Python application and MySQL database. This Oracle Corporation library facilitates seamless communication by providing Python-based access to MySQL database operations. It enables the execution of SQL queries, transaction processing, and database management commands directly from the Python codebase, ensuring efficient data manipulation and retrieval while maintaining database security and integrity.

Visual Studio Code Version 1.105 serves as the integrated development environment, offering comprehensive code editing, debugging, and version control capabilities. Microsoft Corporation's source code editor provides syntax highlighting for both Python and SQL, integrated terminal access, and extension support that streamlines the development process. Its debugging features enable efficient troubleshooting and code optimization throughout the development lifecycle.

Together, these tools form a cohesive development ecosystem that supports the creation of a robust, scalable, and maintainable database management system with integrated financial processing capabilities.

Game_Database.py Implimentation Details

The Core Application for comprehensive player data and financial transaction management, featuring a hierarchical menu system for database operations, virtual economy control, and system administration tasks.

Functions :

Core functions in Game_Database.py handle database operations, input validation, and transactions. They ensure data integrity, user-friendly interaction, and system reliability.

def Recreate_Database():

This function handles complete database initialization and reset with administrative password protection. It removes any existing database and creates a fresh schema with three interconnected tables: PLAYER_DATA for user accounts, CURRENCY_MANAGE for financial operations, and ITEM_SHOP for product management. It establishes foreign key relationships with cascade operations and pre-populates the item shop with standard currency packages.

def Inplen_Validate(comment, minl, maxl, dtype, brkr):

This input validation system acts as the primary gatekeeper for all user interactions. It uses continuous loops to ensure valid input, checking string length for text inputs and numerical ranges for integers. The function includes an exit mechanism that allows operation cancellation via blank input when enabled, maintaining data integrity while ensuring user-friendly interaction.

- **comment:** Prompt message for user input
- **minl:** Minimum input length requirement
- **maxl:** Maximum input length limit
- **dtype:** Data type specification ("str" or "int")
- **brkr:** Boolean flag for exit permission

def acc_inp(n, j):

This email validation function implements a two-stage verification process. It first checks input length using Inplen_Validate, then verifies domain authenticity against approved email providers. The function uses substring search to confirm legitimate domain suffixes, rejecting improperly formatted emails while guiding users toward acceptable formats.

- **n:** Custom name for account prompt
- **j:** Boolean flag for exit permission

def date_valid():

This date validation mechanism leverages MySQL's native parsing capabilities through temporary sandbox tables. It inserts user-input dates into DATE-type columns, relying on MySQL's parser to reject invalid entries. For valid dates, it uses CASE-WHEN logic to verify historical ranges and ensure dates don't exceed the current date.

def update_object(x, y, z):

This database update layer simplifies modification operations by dynamically constructing SQL UPDATE statements using Python string formatting. It executes commands through the database cursor, allowing the MySQL server to locate records via primary key indexes and modify specified column values while maintaining transaction integrity.

- **x:** Column name to update
- **y:** New value for the column
- **z:** UID of the record to modify

def Update_Transaction(packid, uid):

This financial processor handles virtual currency purchases by retrieving item details from ITEM_SHOP and executing atomic SQL updates. It simultaneously increases MONEY_USED by package cost and increments the appropriate currency field by package amount, ensuring perfect synchronization between balances and expenditure tracking.

- **packid:** Package ID from ITEM_SHOP table
- **uid:** User ID for transaction processing

def uid_check(ret, stu, com):

This authentication system provides centralized user identification services. It implements continuous validation loops with exit capabilities, verifying six-digit UID format before checking database existence. Based on parameters, it returns either the UID alone or complete user data records, providing clear feedback for invalid UIDs.

- **ret:** Return type specification (1 for UID, 2 for full data)
- **stu:** Boolean flag for exit permission
- **com:** Prompt message for user input

1. Database Operations Menu :

This menu provides comprehensive CRUD functionality for managing player records, including secure data entry with validation and duplicate checking. It ensures data integrity through safety protocols like two-phase deletion and maintains synchronized records across related tables.

1. Enter Data into PLAYER_DATA :

Handles complete player registration by systematically collecting and validating all required user information including unique six-digit user ID, username with strict length validation, secure password, and properly formatted email account. The system performs rigorous duplicate checking across all unique fields and automatically creates synchronized entries in both player data and currency management tables, establishing complete user profiles with initialized financial accounts.

2. Show Entire PLAYER_DATA TABLE :

Executes a comprehensive SELECT query to retrieve all player records from the database, processing the result set as a structured list of tuples where each tuple represents an individual player's complete data record. The display mechanism systematically presents each field in an organized tabular format that maintains clear column alignment, providing administrators with a complete overview of all registered users for quick reference and data verification.

3. Select Specific Data from PLAYER_DATA :

Enables precise record retrieval using the user ID as the primary search key, beginning with UID format and range validation before executing a focused database query. The system employs robust existence checking by verifying if queries return valid records or null values, displaying complete player profiles when matches are found while providing clear feedback for non-existent UIDs to prevent user confusion.

4. Update PLAYER_DATA Record :

Provides secure modification capabilities through multiple update pathways including username changes with comprehensive duplicate checking, password modifications requiring current password verification, and account updates combining email format validation with system-wide duplicate checking. Each path incorporates cancellation options and executes successful modifications through a centralized update function that maintains database integrity while providing confirmation feedback.

5. Delete Record from PLAYER_DATA :

Implements a safety-focused two-phase deletion protocol beginning with retrieval of all existing user IDs followed by linear search confirmation of target UID existence. Only after successful verification does the system proceed to physical deletion, utilizing the database's primary key index for rapid record removal while automatically handling related currency records through foreign key cascade operations.

2. Manage Player Transactions :

This module handles all virtual economy operations including currency purchases, conversions, and financial tracking. It implements real-time balance updates and generates detailed revenue reports through sophisticated database queries.

1. Buy Coins :

Presents available coin packages dynamically loaded from the item shop database with clear display of quantities and real-money pricing. The system validates user identity through UID verification before processing transactions that simultaneously increase coin balances and update money spent tracking, maintaining perfect financial consistency.

2. Buy Gems :

Mirrors the coin purchase functionality while handling the premium currency type, displaying gem packages with corresponding pricing structures. The process includes robust user authentication and processes transactions using the same engine but applies changes to gem balance fields while maintaining consistent financial tracking.

3. Convert Gems to Coins :

Implements currency exchange with a fixed conversion rate where one gem equals ten coins, beginning with user authentication and available balance verification. The system executes precise mathematical calculations and atomic updates that decrease gem balances while increasing coin balances, maintaining perfect financial equilibrium.

4. Coin and Gem Earning through Gameplay :

Manages achievement tracking through structured input interfaces with implemented limits to prevent data entry errors. The system validates inputs against boundaries before processing updates that increase respective currency balances, providing immediate rewards for gameplay accomplishments.

5. Show Player Transaction Record and Total Earning :

Generates comprehensive financial reports by combining data from multiple tables through sophisticated JOIN operations. Displays individual player balances and calculates aggregate platform revenue using SQL SUM aggregation, offering valuable business intelligence insights.

4. Database Backup and Restore :

Saves multiple tables via serialization into files. Restores data with error handling and integrity checks.

1. Backup Database into File :

Creates complete database snapshots by extracting and serializing information from both main tables into a nested list structure that maintains relational integrity. The system employs serialization to convert data into compact binary format stored in portable archive files.

2. Use Backup File to Restore Database :

Reverses the backup process by reading serialized data and reconstructing the original nested list structure. The system processes each table's data separately with sophisticated error handling that enables partial data recovery when complete restoration isn't possible.

5. Option 4, 5 and 6:

Allows secure database reset and graceful program control. Manages safe shutdowns and environment setup.

4. Recreate GAME_DATA Database :

Provides complete environment reset with administrative password confirmation to prevent accidental execution. Performs destructive then constructive database reconstruction with proper table relationships and pre-populated item catalog.

5. Restart Program :

Provides soft reset capability by manipulating main loop control variables to return navigation to the main menu while preserving database connections and unsaved operational data.

6. Exit Program :

Handles graceful termination by setting loop control variables to trigger natural exit, explicitly closing database connections, and displaying courteous exit messages.

Conclusion

This project demonstrates a comprehensive menu-driven database management system with integrated financial processing capabilities. The system establishes a robust foundation through secure database initialization, creating interconnected tables for user accounts, financial operations, and product management. Its core functionality includes sophisticated data handling with multi-layered validation, efficient CRUD operations, and a complete virtual economy system featuring currency conversion, purchase transactions, and real-time revenue calculation. The implementation extends to backup utilities, financial reporting, and hierarchical menu navigation, showcasing professional-grade architecture suitable for gaming platforms and e-commerce applications. The project successfully combines robust data management with practical business logic, resulting in a scalable solution that addresses essential requirements for modern database systems and financial transaction processing.

Bibliography

Books Referred to:

1. Computer Science with Python XII - NCERT
2. Computer Science with Python XII - Sumita Arora
3. MySQL Cookbookby - Paul DuBois
4. Python Crash Course - Eric Matthes

Websites and Online Resources Referred to:

1. Python Software Foundation Documentation: <https://docs.python.org/3/>
2. MySQL Official Documentation: <https://dev.mysql.com/doc/connector-python/en/>
3. GeeksforGeeks: <https://www.geeksforgeeks.org/python-programming-language/>
4. W3Schools: <https://www.w3schools.com/>